

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – V

Roll No.: T025	Name: Afsha Amjad Mujawar
Class: TYIT	Batch:1
Date of Assignment:28-07-2026	Date/Time of Submission: 28-07-2026

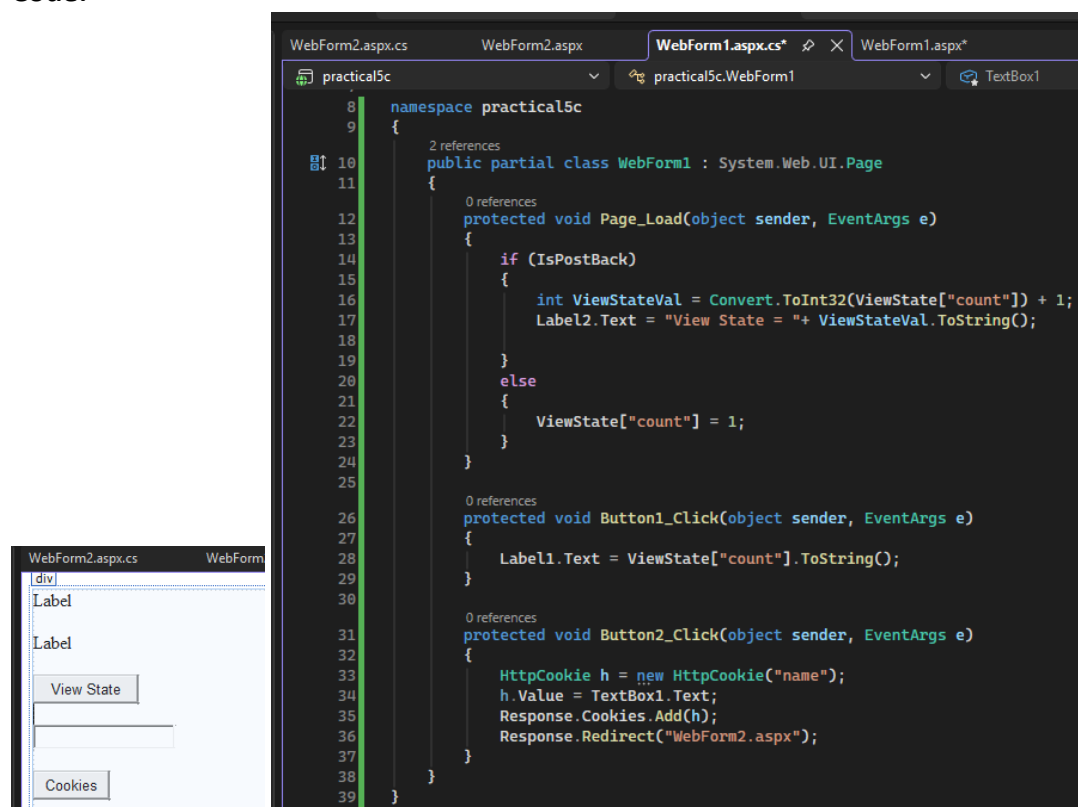
a) Create a web application to demonstrate the use of different types of Cookies.

Algorithm:

Steps:

1. Create Project: Open Visual Studio > Create new ASP.NET Web Application (.NET Framework) > Select Web Forms.
2. Design WebForm1: Add TextBox1, Button1 (ViewState counter), Button2 (Save Cookie & Redirect), Label1, and Label2.
3. Write Code for WebForm1: In Page_Load, track and increment ViewState["count"]. In Button2_Click, create HttpCookie("name"), store value, and redirect.
4. Design & Code WebForm2: Add code in Page_Load to check for Request.Cookies["name"] and display greeting.
5. Run Project: Set WebForm1.aspx as Start Page and press F5 to test.

Code:



```

8 namespace practical5c
9 {
10     public partial class WebForm1 : System.Web.UI.Page
11     {
12         protected void Page_Load(object sender, EventArgs e)
13         {
14             if (IsPostBack)
15             {
16                 int ViewStateVal = Convert.ToInt32(ViewState["count"]) + 1;
17                 Label2.Text = "View State = " + ViewStateVal.ToString();
18             }
19             else
20             {
21                 ViewState["count"] = 1;
22             }
23         }
24     }
25
26     protected void Button1_Click(object sender, EventArgs e)
27     {
28         Label1.Text = ViewState["count"].ToString();
29     }
30
31     protected void Button2_Click(object sender, EventArgs e)
32     {
33         HttpCookie h = new HttpCookie("name");
34         h.Value = TextBox1.Text;
35         Response.Cookies.Add(h);
36         Response.Redirect("WebForm2.aspx");
37     }
38 }
39
  
```

```

1 <%@ Page Language="C#" AutoEventWireup="true" CodeBehind="WebForm2.aspx.cs" Inherits="practical5c.W
2
3 <!DOCTYPE html>
4
5 <html xmlns="http://www.w3.org/1999/xhtml">
6 <head runat="server">
7 <title></title>
8 </head>
9 <body>
10 <form id="form1" runat="server">
11 <div>
12 <asp:Label ID="Label1" runat="server" Text="Label"></asp:Label>
13
14 </div>
15 </form>
16 </body>
17 </html>

```

```

1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Web;
5 using System.Web.UI;
6 using System.Web.UI.WebControls;
7
8 namespace practical5c
9 {
10     2 references
11     public partial class WebForm2 : System.Web.UI.Page
12     {
13         0 references
14         protected void Page_Load(object sender, EventArgs e)
15         {
16             if (Request.Cookies["name"] != null)
17             {
18                 Response.Write("Welcome " + Request.Cookies["name"].Value);
19             }
20         }
21     }
22 }

```

Output:

1

View State = 2

View State

afsha

Cookies

Welcome Afsha Mujawar
Label

b) Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.

Algorithm:

1. Create Project: Open Visual Studio Create new ASP.NET Core Web App (MVC).
2. Install Packages: Open Package Manager Console and run the necessary Entity Framework Core commands.
3. Install-Package Microsoft.EntityFrameworkCore.SqlServer
4. Install-Package Microsoft.EntityFrameworkCore.Tools
5. Define Model & DbContext: Create Product.cs (Id, Name, Price) and ApplicationDbContext.cs containing DbSet<Product> Products.
6. Configure Database Connection: Add connection string in appsettings.json and register AddDbContext in Program.cs.
- 7.

Code:USE ProductDB;

No need to create the table manually. EF Core will create it.

Step 1: Product Model (Product.cs)

```

1 namespace prcatical5.Models
2 {
3     public class Product
4     {
5         public int Id { get; set; }
6         public string Name { get; set; }
7         public int Price { get; set; }
8     }
9 }
10
11

```

Step 3: ApplicationDbContext (AppDbContext.cs)

```

1 using Microsoft.EntityFrameworkCore;
2 using prcatical5.Models;
3 public class ApplicationDbContext : DbContext
4 {
5     public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options):base (options)
6     {
7     }
8     public DbSet<Product> Products { get; set; }
9 }

```

Step 4: appsettings.json

```

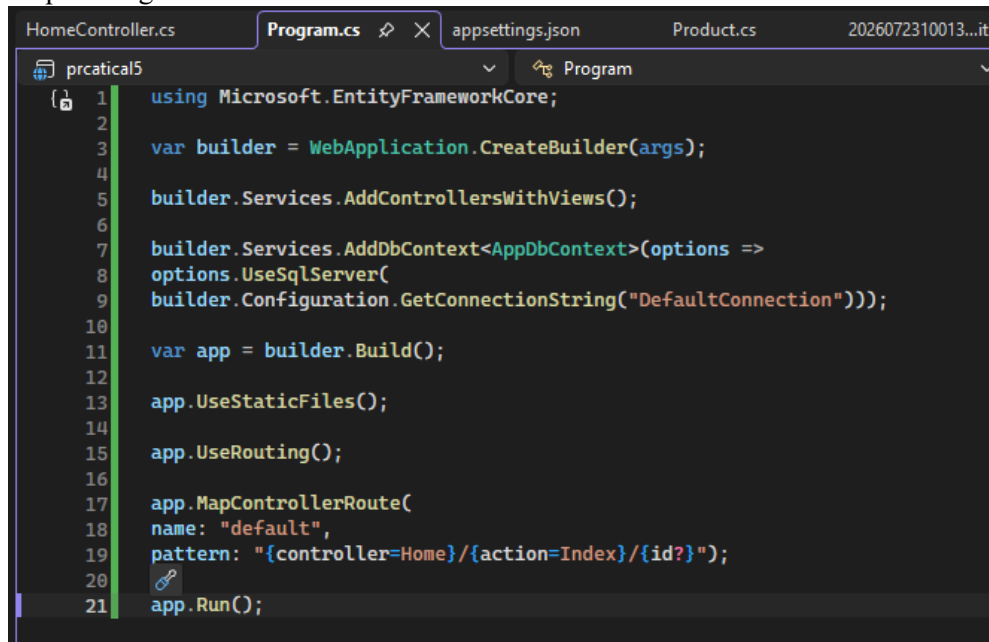
1 {
2     "Logging": {
3         "LogLevel": {
4             "Default": "Information",
5             "Microsoft.AspNetCore": "Warning"
6         }
7     },
8     "AllowedHosts": "*",
9     "ConnectionStrings": {
10        "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=ProductDB;Trusted_Connection=True;TrustServerCertificate=True"
11    }
12 }
13

```

If you use SQL Server Express:

"Server=.\SQLEXPRESS;Database=ProductDB;Trusted_Connection=True;TrustServerCertificate=True;"

Step 5: Program.cs



```

1  using Microsoft.EntityFrameworkCore;
2
3  var builder = WebApplication.CreateBuilder(args);
4
5  builder.Services.AddControllersWithViews();
6
7  builder.Services.AddDbContext<AppDbContext>(options =>
8  options.UseSqlServer(
9  builder.Configuration.GetConnectionString("DefaultConnection")));
10
11 var app = builder.Build();
12
13 app.UseStaticFiles();
14
15 app.UseRouting();
16
17 app.MapControllerRoute(
18 name: "default",
19 pattern: "{controller=Home}/{action=Index}/{id?}");
20
21 app.Run();
  
```

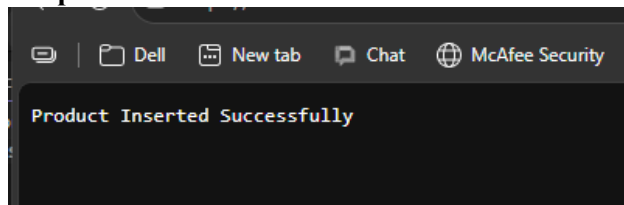
Step 7: Install Packages ,Open Package Manager Console
 Install-Package Microsoft.EntityFrameworkCore.SqlServer
 Install-Package Microsoft.EntityFrameworkCore.Tools

Step 8: Migration
 Add-Migration InitialCreate
 Update-Database

Step 9: Run
 Run the project and open:
<https://localhost:xxxx/Home/Index>
 Output:
 Product Inserted Successfully

Step 10: Verify
 Open SQL Server and run:
 SELECT * FROM Products;
 Output:
 Id Name Price
 1 Laptop 50000

Output:



	Id	Name	Price
1	1	Laptop	50000

c) Develop a web application to perform CRUD Operations (Create, Read, Update, Delete) on Employee table using EF Core.

Algorithm:

Steps :

1. Define Model: Create Employee.cs with properties (Id, Name, Department, Salary).
2. Setup DbContext: Create AppDbContext.cs with DbSet<Employee> Employees and register it inside Program.cs.
3. Apply Migrations: Run Add-Migration AddEmployeeTable followed by Update-Database in Package Manager Console.
4. Build Controller Actions: In EmployeesController.cs, implement methods for Create, Read, Update, and Delete operations.
5. Create Views: Generate corresponding Razor views under Views/Employees/ (Index, Create, Edit, Delete).

Code: Employee.cs

```
namespace EmployeeCRUD.Models
{
    public class Employee
    {
        public int Id { get; set; }
        public required string Name { get; set; }

        public required string Department { get; set; }

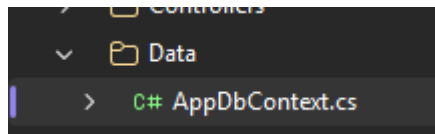
        public double Salary { get; set; }
    }
}
```

Program.cs

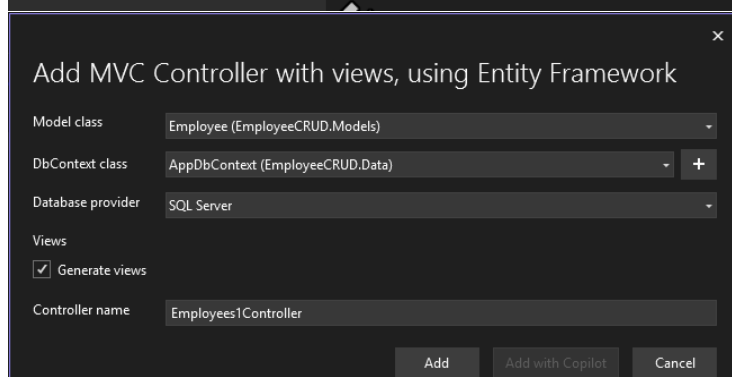
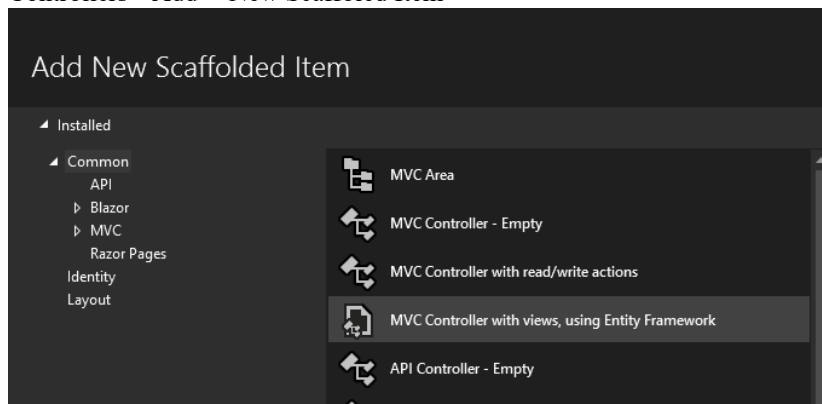
```
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Data;
builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("MyConnection")));
appsettings.json:
"AllowedHosts": "*",
"ConnectionStrings": {
    "MyConnection":
"Server=(localdb)\MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;TrustServerCertificate=True",
    "AppDbContext":
"Server=(localdb)\mssqllocaldb;Database=AppDbContext;Trusted_Connection=True;MultipleActiveResultSets=true"
}
```

AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Models;
namespace EmployeeCRUD.Data
{
    public class AppDbContext:DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
        public DbSet <Employee> Employees { get; set; }
    }
}
```



Controllers->Add-> New Scaffolded Item

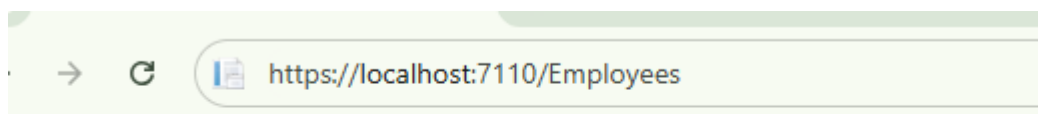


Output:

EmployeeCRUD Home Privacy

Welcome

Learn about [building Web apps with ASP.NET Core](#).



EmployeeCRUD Home Privacy

EmployeeCRUD Home Privacy

Create

Employee

Name

Department

Salary

Create

[Back to List](#)

EmployeeCRUD Home Privacy			
Index			
Create New			
Name	Department	Salary	
Akansha	IT	90000	Edit Details Delete

Output:

d) Authentication and Authorization in ASP.NET Core.

Algorithm:

Steps :

1. Register Auth Services: In Program.cs, add services using `AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme).AddCookie(...)` and `AddAuthorization()`.
2. Configure Pipeline: Add `app.UseAuthentication()` before `app.UseAuthorization()` in Program.cs.
3. Build Login Logic: In HomeController.cs, validate user credentials, create a ClaimsIdentity, and call `await HttpContext.SignInAsync(...)`.
4. Protect Route: Add the `[Authorize]` attribute directly above the `Secure()` action method.
5. Build View & Logout: Create Views/Home/Index.cshtml with a login form, and implement `await HttpContext.SignOutAsync(...)` inside the `Logout()` action.

Code:

Program.cs:

```
using Microsoft.AspNetCore.Authentication.Cookies;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Home/Index";
    });
builder.Services.AddAuthorization();
var app = builder.Build();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();
```

homecontroller.cs:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using System.Security.Claims;
namespace AuthDemo.Controllers
{
    0 references
    public class HomeController : Controller
    {
        0 references
        public IActionResult Index()
        {
            return View();
        }
        [HttpPost]
```

```

public async Task<IActionResult> Index(string username, string password)
{
    if (username == "Nandini" && password == "2006")
    {
        var claims = new List<Claim>
        {
            new Claim(ClaimTypes.Name, username)
        };
        var identity = new ClaimsIdentity(claims,
            CookieAuthenticationDefaults.AuthenticationScheme);
        await HttpContext.SignInAsync(
            CookieAuthenticationDefaults.AuthenticationScheme,
            new ClaimsPrincipal(identity));
        return RedirectToAction("Secure");
    }
    ViewBag.Message = "Invalid Username or Password";
    return View();
}
[Authorize]

```

```

public IActionResult Secure()
{
    return Content("Welcome! Login Successful.");
}
0 references
public async Task<IActionResult> Logout()
{
    await HttpContext.SignOutAsync(
        CookieAuthenticationDefaults.AuthenticationScheme);
    return RedirectToAction("Index");
}
}

```

index.cshtml

```

<h2>Login</h2>
<form method="post">
    Username :
    <input type="text" name="username" /><br /><br />
    Password :
    <input type="password" name="password" /><br /><br />
    <input type="submit" value="Login" />
</form>
<p style="color: red">
    @ViewBag.Message
</p>

```

Output:

Login

Username:

Password: 

Welcome! Login Successful.